STUDENT SCHOLARSHIP APPLICATION AND DISBURSEMENT TRACKER

A record for every application, the stage it has reached, and how long it has been waiting there.

SIH 2026 Internal Practical Assessment · VAISHNAVI M · Reg 411724205053 · PSVPEC · IT · Year III

THE PROBLEM

A student cannot be told where their application has reached

Scholarship applications move through a clerk's register, a document check, a section review, and a sanction step - and each stage is recorded on a separate piece of paper or a separate register.

The result: applications quietly stall at one stage for months, and nobody in the section notices until a student comes and asks.

Who is affected

Students

waiting on money they've already been told they're eligible for

Scholarship section staff

who have no easy way to see what's stuck

Eligible-but-needy students most of all

- the ones least able to keep chasing the office

A simple web tracker that records every scholarship application, shows exactly how long it has been sitting at its current stage, and lets any student's status be answered in one search.

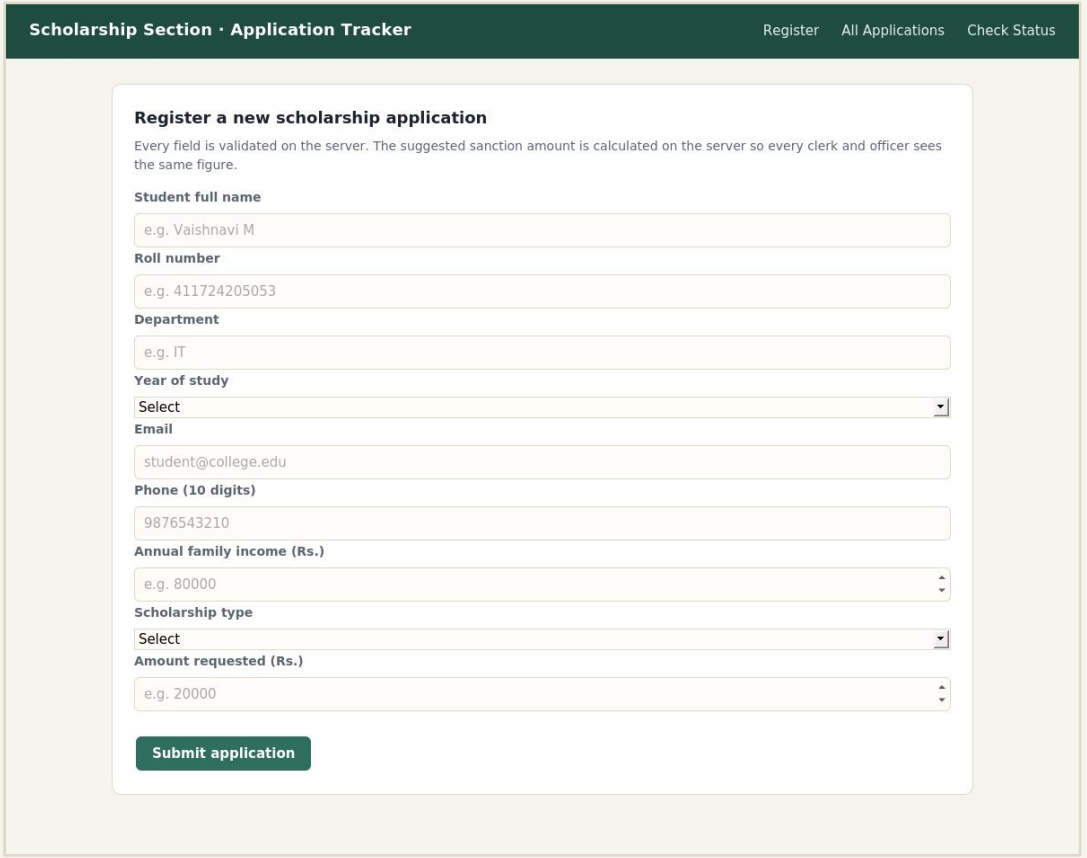
THE SYSTEM

Register screen - one action, start to finish

Clerk fills the form -> server validates every field -> suggested sanction amount is calculated on the server -> saved to SQLite -> shown back on screen.

Stack: HTML/CSS/JS frontend + Flask (Python) backend + SQLite database

4 tables: students, scholarship_applications, application_stage_history (keeps every stage change, never overwritten), document_verifications.



Screenshot: own working solution (index.html + Flask API)

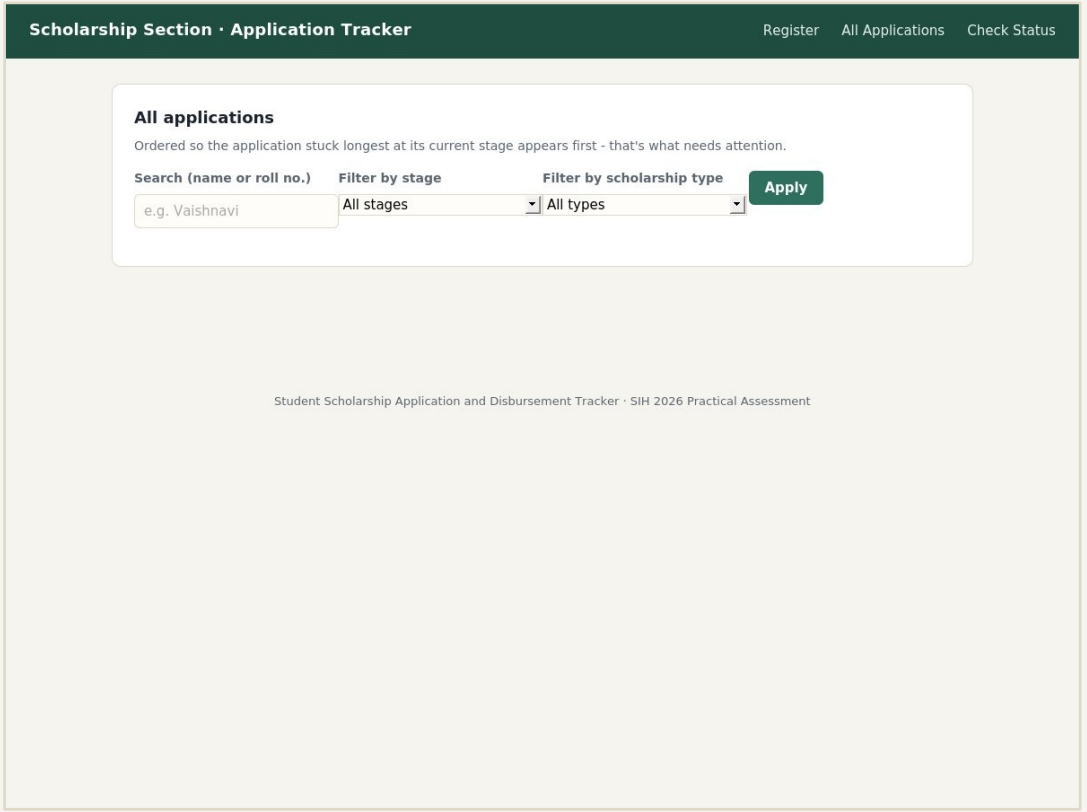
THE SYSTEM

Listing - what needs attention appears first

Applications are ordered by days spent at the current stage (longest first), with search and stage/type filters, and the result count always shown.

How the suggested sanction amount is calculated (server-side)

Annual family income	Suggested sanction
< Rs. 1,00,000	100% of amount requested
Rs. 1,00,000 - 2,49,999	75% of amount requested
>= Rs. 2,50,000	50% of amount requested



Screenshot: own working solution (list.html)

THE CLASSIFIER

Document photo quality check

80 labelled document-photo images (clear / unclear), generated with controlled blur, brightness and noise, split by the underlying document so the same document never appears on both sides.

Approach: fine-tune a pretrained MobileNetV2 (ImageNet weights) - only the final classification layer is retrained, not the whole network from scratch.

Honest note on this build

The development sandbox used to build this project blocks downloads from download.pytorch.org, so the pretrained ImageNet weights could not be fetched here - the shipped model fell back to random initialisation, and its demo accuracy is low as a result. train.py is written correctly for real transfer learning: running it on a normal machine with internet access downloads the real pretrained weights automatically and needs no code change. Low-confidence predictions are never forced either way - they're left blank for manual review.

STATUS

What works, and what's left

Working end to end

- ✓ Register -> validate -> calculate -> store -> display
- ✓ 4-table schema with a full stage-history table
- ✓ Classifier pipeline: dataset, source-disjoint split, fine-tuning, confidence threshold
- ✓ Search, filter, and "stuck longest" ordering on the listing screen
- ✓ 23/23 automated integration tests passing (main flow, every invalid insert, empty/error states)

Not yet done / next step

- → No login / role-based access yet - anyone with the URL can act as clerk or officer
- → Classifier needs a real internet connection at training time to use true pretrained weights (see previous slide)
- → Next improvement: add authentication so only the right role can advance a stage

Thank you

Full source, README, ER diagram, tests, and this presentation are in one public GitHub repository.

VAISHNAVI M · Reg 411724205053 · PSVPEC · IT · Year III · SIH 2026